

Introducción a las API's en Python, JavaScript y C#

Parte 1: ¿Qué es una API?

Imagina que estás en un restaurante. Tú (el cliente) quieres comer, pero no puedes entrar a la cocina. El mesero (la **API**) es el intermediario que toma tu orden, la lleva a la cocina (el **servidor**) y te trae la comida (los **datos**). La carta (el **menú**) es la documentación que te indica qué puedes pedir y cómo.

En términos técnicos, una **API** (Application Programming Interface) es un conjunto de reglas y definiciones que permite que dos aplicaciones de software se comuniquen entre sí. Las APIs exponen funcionalidades y datos de forma controlada y segura.

¿Para qué sirven?

- Obtener datos actualizados (clima, noticias, precios).
- Integrar servicios externos (pagos con tarjeta, mapas).
- Conectar aplicaciones (publicar en redes sociales automáticamente).

Parte 2: Conceptos Clave

2.1 Endpoint

Es una **URL concreta** que identifica un recurso específico dentro de la API. Por ejemplo, para obtener el pronóstico del tiempo de Open-Meteo, el endpoint es: <https://api.open-meteo.com/v1/forecast>

2.2 Parámetros (Query Strings)

Son pares **clave=valor** que se añaden después del signo **?** en la URL para afinar la petición. Por ejemplo: [?latitude=40.42&longitude=-3.70¤t_weather=true](https://api.open-meteo.com/v1/forecast?latitude=40.42&longitude=-3.70¤t_weather=true) Esto indica que queremos el clima actual en las coordenadas de Madrid.

2.3 Métodos HTTP

Indican la acción a realizar. Los más comunes son:

- **GET** → Obtener datos (el que usaremos).
- **POST** → Crear un nuevo recurso.
- **PUT / PATCH** → Actualizar.
- **DELETE** → Eliminar.

2.4 Códigos de Estado HTTP

El servidor responde con un código numérico que indica el resultado:

- **200** OK: Todo bien.
- **201** Created: Recurso creado.
- **400** Bad Request: Petición mal formada.
- **401** Unauthorized: Falta autenticación.
- **404** Not Found: Recurso no existe.
- **429** Too Many Requests: Límite de peticiones excedido.
- **500** Internal Server Error: Error del servidor.

2.5 API Key

Es una clave única que identifica a la aplicación que hace la petición. Se envía normalmente en un **header** (cabecera) de la petición, por ejemplo: **X-API-Key: tu_clave_secreta** Algunas APIs (como Open-Meteo) no requieren clave, lo que las hace ideales para aprender.

2.6 JSON (JavaScript Object Notation)

Es el formato estándar para intercambiar datos. Una respuesta JSON típica se ve así:

```
"latitude": 40.42,  
"longitude": -3.7,  
"current_weather": {  
  "temperature": 12.3,  
  "windspeed": 11.2,  
  "weathercode": 0  
}  
}
```

Parte 3: Explorando el Directorio "Free APIs"

El sitio [Free APIs](#) contiene cientos de APIs públicas organizadas por categorías: Clima, Noticias, Mascotas, etc. Puedes explorarlas para encontrar datos para tus proyectos.

Nosotros usaremos la API de **Open-Meteo** (categoría Weather) por su simplicidad y porque no necesita clave.

Parte 4: Construyendo una Aplicación del Clima

Vamos a crear una pequeña aplicación que consulte el clima actual en Madrid (coordenadas 40.42, -3.70) y muestre la temperatura y la velocidad del viento.

4.1 La Petición que Haremos

Endpoint: <https://api.open-meteo.com/v1/forecast>

Parámetros:

- `latitude=40.42`
- `longitude=-3.70`
- `current_weather=true`

URL completa: `https://api.open-meteo.com/v1/forecast?latitude=40.42&longitude=-3.70¤t_weather=true`

4.2 Explicación del Código (Común a los tres lenguajes)

1. **Importar librerías** necesarias para hacer peticiones HTTP.
2. **Definir la URL y los parámetros.**
3. **Realizar la petición GET.**
4. **Verificar el código de estado** (si no es 200, manejar el error).
5. **Parsear la respuesta JSON.**
6. **Extraer los datos** de temperatura y viento.
7. **Mostrarlos** en la consola o en la página.

A continuación, presentamos el mismo programa en **Python**, **JavaScript** (para navegador) y **C#** (consola).

Parte 5: Explicación Detallada del Código

5.1 Python

- Usamos la librería `requests`, que simplifica las peticiones HTTP.
- `requests.get()` acepta un diccionario `params` y construye la URL automáticamente.
- `raise_for_status()` lanza una excepción si el código de estado es 4xx o 5xx.
- `respuesta.json()` convierte la respuesta en un diccionario Python.
- Accedemos a los datos anidados con `datos["current_weather"]["temperature"]`.

```
#####
#CÓDIGO PYTHON#
#####
import requests

def obtener_clima():
    # 1. Definir URL y parámetros
    url = "https://api.open-meteo.com/v1/forecast"
    params = {
        "latitude": 40.42,
        "longitude": -3.70,
        "current_weather": True
    }

    try:
        # 2. Hacer la petición GET
        respuesta = requests.get(url, params=params)
        # 3. Verificar si hubo error (lanza excepción si código != 200)
```

```

respuesta.raise_for_status()

# 4. Convertir la respuesta a diccionario Python
datos = respuesta.json()

# 5. Extraer datos del clima actual
clima_actual = datos["current_weather"]
temperatura = clima_actual["temperature"]
viento = clima_actual["windspeed"]

# 6. Mostrar resultados
print("Clima en Madrid:")
print(f"Temperatura: {temperatura} °C")
print(f"Viento: {viento} km/h")

except requests.exceptions.RequestException as e:
    print(f"Error en la petición: {e}")

if __name__ == "__main__":
    obtener_clima()

```

5.2 JavaScript

- `fetch()` es la función nativa para peticiones HTTP en el navegador.
- `URLSearchParams` construye la cadena de consulta de forma segura.
- `await` espera la respuesta; `respuesta.ok` es `true` para códigos 200-299.
- `respuesta.json()` devuelve una promesa con el objeto JavaScript.
- Manipulamos el DOM para mostrar los resultados.

```

// clima.js - Para usar en un navegador con HTML
async function obtenerClima() {
    const url = "https://api.open-meteo.com/v1/forecast";
    const params = new URLSearchParams({
        latitude: 40.42,
        longitude: -3.70,
        current_weather: true
    });

    try {
        // Hacer la petición fetch
        const respuesta = await fetch(`${url}?${params}`);

        // Verificar si la respuesta es correcta (código 200-299)
        if (!respuesta.ok) {
            throw new Error(`Error HTTP: ${respuesta.status}`);
        }

        // Convertir a JSON
        const datos = await respuesta.json();
    }
}

```

```

// Extraer datos
const temperatura = datos.current_weather.temperature;
const viento = datos.current_weather.windspeed;

// Mostrar en el elemento HTML (suponiendo un div con id="resultado")
document.getElementById("resultado").innerHTML = `
    <p><strong>Clima en Madrid:</strong></p>
    <p>Temperatura: ${temperatura} °C</p>
    <p>Viento: ${viento} km/h</p>
`;
} catch (error) {
    console.error("Error:", error);
    document.getElementById("resultado").innerText = "Error al obtener el
clima.";
}
}

// Llamar a la función cuando la página cargue
window.onload = obtenerClima;

```html

<!DOCTYPE html>
<html>
<head>
 <title>Clima en Madrid</title>
</head>
<body>
 <h1>Clima en Madrid</h1>
 <div id="resultado">Cargando...</div>
 <script src="clima.js"></script>
</body>
</html>

```

### 5.3 C#

- `HttpClient` es la clase recomendada para peticiones HTTP en .NET.
- `GetAsync` realiza la petición de forma asíncrona.
- `EnsureSuccessStatusCode()` lanza excepción si el código no es exitoso.
- `ReadAsStringAsync()` lee el cuerpo como texto.
- `JsonSerializer.Deserialize<>()` convierte el JSON a objetos C# (requiere clases definidas previamente).

```

using System;
using System.Net.Http;
using System.Text.Json;
using System.Threading.Tasks;

```

```

namespace ClimaApp
{
 // Clases para mapear la respuesta JSON (solo las propiedades que nos
 interesan)
 public class ClimaResponse
 {
 public CurrentWeather current_weather { get; set; }
 }

 public class CurrentWeather
 {
 public float temperature { get; set; }
 public float windspeed { get; set; }
 }

 class Program
 {
 static async Task Main(string[] args)
 {
 using HttpClient client = new HttpClient();

 string url = "https://api.open-meteo.com/v1/forecast?
latitude=40.42&longitude=-3.70¤t_weather=true";

 try
 {
 // Hacer petición GET
 HttpResponseMessage response = await client.GetAsync(url);
 response.EnsureSuccessStatusCode(); // Lanza excepción si no es
exitoso

 // Leer contenido como string
 string jsonString = await response.Content.ReadAsStringAsync();

 // Deserializar JSON a objeto C#
 var options = new JsonSerializerOptions {
PropertyNameCaseInsensitive = true };
 ClimaResponse datos = JsonSerializer.Deserialize<ClimaResponse>
(jsonString, options);

 // Mostrar resultados
 Console.WriteLine("Clima en Madrid:");
 Console.WriteLine($"Temperatura:
{datos.current_weather.temperature} °C");
 Console.WriteLine($"Viento: {datos.current_weather.windspeed}
km/h");
 }
 catch (HttpRequestException e)
 {
 Console.WriteLine($"Error en la petición: {e.Message}");
 }
 }
 }
}

```

```
}
}
```

---

## Parte 6: Ampliando el Ejemplo - Usar una API con Key

---

Muchas APIs requieren autenticación. Por ejemplo, **NewsAPI** ([newsapi.org](https://newsapi.org)) necesita una clave. Modifiquemos nuestro programa en Python para obtener noticias de tecnología usando una clave.

```
import requests

API_KEY = "tu_clave_aqui" # Reemplaza con tu clave real

url = "https://newsapi.org/v2/top-headlines"

params = {
 "category": "technology",
 "country": "us"
}
headers = {"X-Api-Key": API_KEY}

respuesta = requests.get(url, params=params, headers=headers)
if respuesta.status_code == 200:
 datos = respuesta.json()
 for articulo in datos["articles"][:3]:
 print(articulo["title"])
else:
 print(f"Error {respuesta.status_code}: {respuesta.text}")
```

Observa cómo la clave se envía en el header **X-Api-Key**. Nunca debes incluir claves directamente en el código si este será público (especialmente en JavaScript del lado del cliente). En su lugar, usa variables de entorno o un backend.

---

## Parte 7: Buenas Prácticas y Consejos

---

1. **Lee siempre la documentación** de la API que vayas a usar.
2. **Maneja los errores** adecuadamente (usa **try/catch** o verifica códigos de estado).
3. **No expongas claves** en código cliente (JavaScript en navegador). Úsalas desde un servidor.
4. **Respetar los límites de tasa** (rate limiting). Si recibes un error 429, espera antes de reintentar.
5. **Usa variables de entorno** para guardar claves en tus aplicaciones backend.
6. **Prueba las APIs con herramientas como Postman** antes de escribir código.

---

## Parte 8: Conclusión

---

---

Has aprendido qué es una API, sus conceptos fundamentales y has creado una pequeña aplicación funcional en tres lenguajes diferentes. Ahora puedes explorar el directorio **Free APIs** y construir tus propios proyectos integrando datos del mundo real.

¡El cielo es el límite! O mejor dicho, ¡el límite es la documentación de la API!